HTTPS & SSH

MODULE 4

Prepared by

Jesna J S

Assistant Professor CSE, TKMCE

HTTPS (HTTP over SSL/TLS)

- Refers to the **combination of HTTP and SSL/TLS to implement secure communication between a Web browser and a Web server**.
- Built into all modern Web browsers.
- Its use depends on the Web server supporting HTTPS communication.
- The principal difference seen by a user of a Web browser is that URL addresses begin with https:// rather than http://.
- A normal HTTP connection uses port 80.
- If HTTPS is specified, port 443 is used, which invokes SSL/TLS.
- HTTPS is documented in RFC 2818

- When HTTPS is used, the following elements of the communication are encrypted:
  - ✓URL of the requested document
  - ✓Contents of the document
  - ✓Contents of browser forms (filled in by browser user)
  - ✓Cookies sent from browser to server and from server to browser
  - ✓Contents of HTTP header
- It has two phases:
  1. **Connection Initiation**
  2. **Connection Closure**

CONNECTION INITIATION

- In HTTPS, the HTTP client also *acts as the TLS client*.
- The client initiates a TCP connection to the server on the correct port.
- It sends a TLS Client_Hello message to start the TLS handshake.
- Once the TLS handshake is completed, the client starts sending HTTP requests over the secured connection.

Three levels of connection awareness in HTTPS:

*When a client (e.g., a web browser) connects to a secure website (**HTTPS**), three key layers work together to establish a **secure** and **reliable** connection*

1. **HTTP Level:** The client requests a connection from the HTTP server.
2. **TLS(Security Layer) Level:** A TLS session is established, which can support multiple connections.
3. **TCP Level:** A TCP connection is created between the client and server before TLS starts.

Step 1: TCP connection (Transport layer)

- The **HTTP client (browser)** first establishes a **TCP connection** to the server.
- This happens on **port 443**
- TCP ensures:
  - ✓Reliable data delivery
  - ✓Ordered transmission
  - ✓Error detection

Step 2: TLS handshake (Security layer)

- After TCP connection is established:
- The **HTTP client now acts as the TLS client**
- It sends a **TLS ClientHello message** to the server
- The **ClientHello** includes:
  - ✓Supported TLS versions
  - ✓Supported cipher suites
  - ✓Random number (for key generation)
- The server responds with:
  - **ServerHello**
  - Digital certificate
  - Key exchange details

- During the TLS handshake:
  - ✓Server authentication happens
  - ✓Encryption keys are generated
  - ✓A secure TLS session is established

Step 3: Secure HTTP communication

- After TLS handshake finishes,
- The client sends **HTTP requests (GET, POST, etc.)**
- These HTTP messages are:
  - Encrypted by TLS
  - Transmitted through TCP

Connection Closure

- To close an HTTP connection, a client/server includes:

  Connection: close

- This means the connection will be closed after delivering the response.
- HTTPS closure requires closing TLS first, which also closes the TCP connection.
- TLS uses the alert protocol to send a **close_notify** alert before terminating the connection.
- A client/server may close the connection without waiting for a close_notify, causing an incomplete close.
- **If a TCP connection is closed without a close_notify or Connection: close, it could be:**
  - ✓A server error.
  - ✓A network issue.
  - ✓A potential attack → The client should issue a security warning.

SSH (SECURE SHELL)

SSH

- SSH (Secure Shell) is a protocol for secure network communication.
- It is designed to be simple and cost-effective to implement.
- **Initially created to replace TELNET, which lacked security.**
- Provides secure remote logins.
- Supports **client/server communication** for:
    - **File transfer (SecureCopyProtocol, SFTP)**
    - **Secure email**
    - **X tunneling (secure GUI applications over a network)**
- SSH1: The first version, focused on remote logins but had security flaws.
- SSH2: Improved version that fixed security vulnerabilities.
    - Documented in IETF RFCs 4250–4256.

- Available on most operating systems.
- Preferred method for remote logins.
- Widely used for encryption outside embedded systems.
- SSH is organized as three protocols that typically run on top of TCP
    1. **Transport Layer Protocol**
    2. **User Authentication Protocol**
    3. **Connection Protocol**

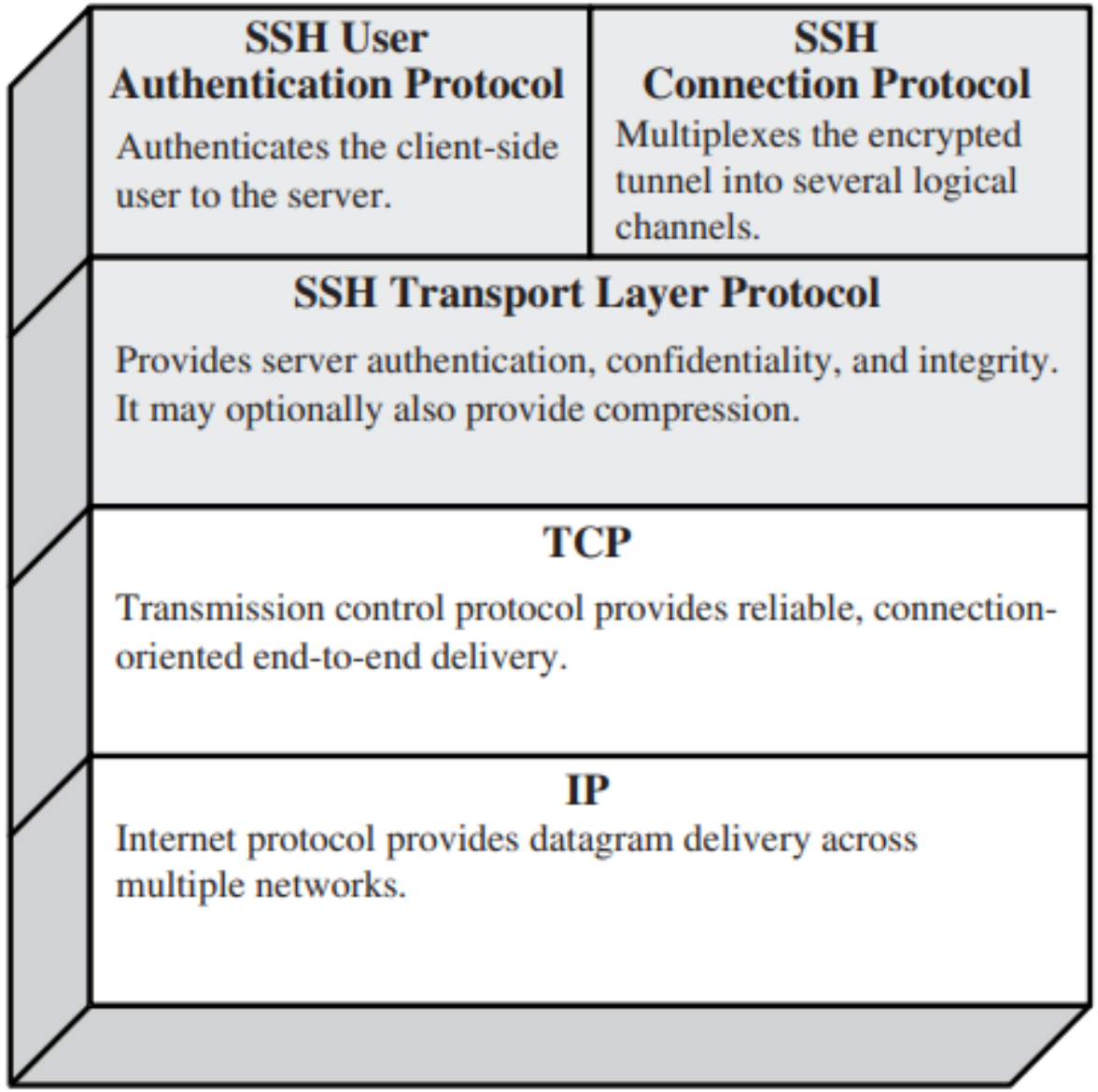

Figure 16.8 SSH Protocol Stack

1. Transport Layer Protocol

- **Provides server authentication, data confidentiality, and data integrity with forward secrecy** (i.e., if a key is compromised during one session, the knowledge does not affect the security of earlier sessions).
- The transport layer may optionally provide compression.
- **Server authentication** happens at the transport layer using **public/private key pairs**.
- The server host key is used during key exchange to authenticate the identity of the host.
- Transport layer ensures data confidentiality by encrypting all transmitted data.
- Data integrity is ensured by use of MAC.
- An important security feature provided by SSH transport layer is forward secrecy[ensures the use of temporary keys].

Key exchange process:

- A **server may have multiple host keys using multiple different asymmetric encryption algorithms**.
- When a client connects:
  - ✓The client tells which algorithms it support.
  - ✓The server selects a matching host key.
- Sometimes, **more than one server machine uses the same SSH host key**.
  - To the client, these machines appears as one logical server, it is used at ***load balanced servers, server clusters and cloud environments***..

2. User Authentication Protocol

- The User Authentication Protocol provides the means by which the client is authenticated to the server.
- The client sends an SSH_MSG_USERAUTH_REQUEST to request authentication.
- The server responds with:
  - ✓**SSH_MSG_USERAUTH_FAILURE** if authentication fails or more methods are needed.
  - ✓**SSH_MSG_USERAUTH_SUCCESS** if authentication succeeds.

SSH_MSG_USERAUTH_REQUEST (50)

- Sent by the client to request authentication.
- Format:
  - ➢byte → Message type (50).
  - ➢string → user name (identity of the client).
  - ➢string → service name (service client wants access to, e.g., SSH).
  - ➢string → method name (authentication method used)
  - ➢.... → Method-specific fields (depends on the authentication method).

SSH_MSG_USERAUTH_FAILURE (51)

- Sent by the **server** if:
  - Authentication **fails**.
  - More authentication methods are required.
- Format:
  - ➢byte → Message type (51).
  - ➢name-list → List of other authentication methods that can be used.
  - ➢boolean → partial success (true if at least one authentication method succeeded)

SSH_MSG_USERAUTH_SUCCESS (52)

- Sent by the server when authentication succeeds.
- Format: byte → Message type (52)

AUTHENTICATION METHODS

- The server may require one or more of the following authentication methods.

1.Publickey:

- The details of this method depend on the chosen public-key algorithm.
- **The client sends its public key to the server, along with a message signed using its private key.**
- Upon receiving the message, the server first verifies if the provided public key is valid for authentication.
- If the key is acceptable, the server then checks whether the signature is correct.

2. Password:

- The client sends a message containing a plaintext password, which is protected by encryption by the Transport Layer Protocol.

3. Hostbased:

*Authentication is **performed on the client's host rather than the client itself**.*

- **Host-based authentication** is a method where:
  - ✓The **server authenticates the client's machine (host)**
  - ✓Not the individual user on that machine
  - ✓If the **host is trusted**, then **all users from that host are trusted**.

3. Connection Protocol

- The **SSH Connection Protocol** is the **top layer** of the SSH protocol stack.
- It runs **on top of the SSH Transport Layer Protocol**
- **The Connection Protocol assumes that a secure connection already exists**
- After the transport layer completes:
  1. Key exchange
  2. Server authentication
  3. Encryption setup
- A secure tunnel is created.
- **This tunnel is an encrypted and authenticated channel between client and server.**
- **The Connection Protocol uses this tunnel to carry all SSH communication.**

Multiplexing logical channels

- The Connection Protocol allows **multiple logical channels to operate over a single secure tunnel** , process is called **multiplexing.**
- **All channels are encrypted together but function independently.**
- **Each task uses a separate channel,**
  - This prevents interference between tasks.
  - Failure or closure of one channel does not affect others.
- **Either the client or the server can request to open a channel.**
  - ✓The other side must accept or reject the request.
  - ✓Channel opening is negotiated securely.
- Each side assigns **a unique channel number**.
  - Channel numbers are: Local identifiers, independent on each side
  - The client's channel number and server's channel number may differ.

- **Each channel uses flow control.**
- **Flow control is implemented using a window size.**
- The window defines **how much data can be sent without acknowledgment**.
- A channel goes through three stages:
  1. Opening the channel
  2. Data transfer
  3. Closing the channel
- To open a new channel, a side assigns a local number and sends a request message.

byte	SSH_MSG_CHANNEL_OPEN
string	channel type
uint32	sender channel
uint32	initial window size
uint32	maximum packet size
....	channel type specific data follows

- If the remote side **successfully opens** the channel, it sends an **SSH_MSG_CHANNEL_OPEN_CONFIRMATION** message.
- This message includes:
  - ✓**Sender channel number**
  - ✓**Recipient channel number**
  - ✓**Window size**
  - ✓**Packet size** for incoming data
- If the channel **cannot be opened**, the remote side sends an **SSH_MSG_CHANNEL_OPEN_FAILURE** message.
- This message includes a **reason code** for the failure.

Data Transfer

- Once the channel is open, data is sent using **SSH_MSG_CHANNEL_DATA** messages.
  - These messages include:
    - **Recipient channel number**
    - **Block of data**
- Data transfer can continue **as long as the channel remains open**.

Closing a Channel

- When either side wants to close the channel, it sends an **SSH_MSG_CHANNEL_CLOSE** message.
  - This message includes the **recipient channel number**.

SSH Packet Exchange (Steps)

1. Identification String Exchange
2. Algorithm Negotiation
3. Key Exchange (Diffie-Hellman)
4. Finalizing Key Exchange
5. Service Request

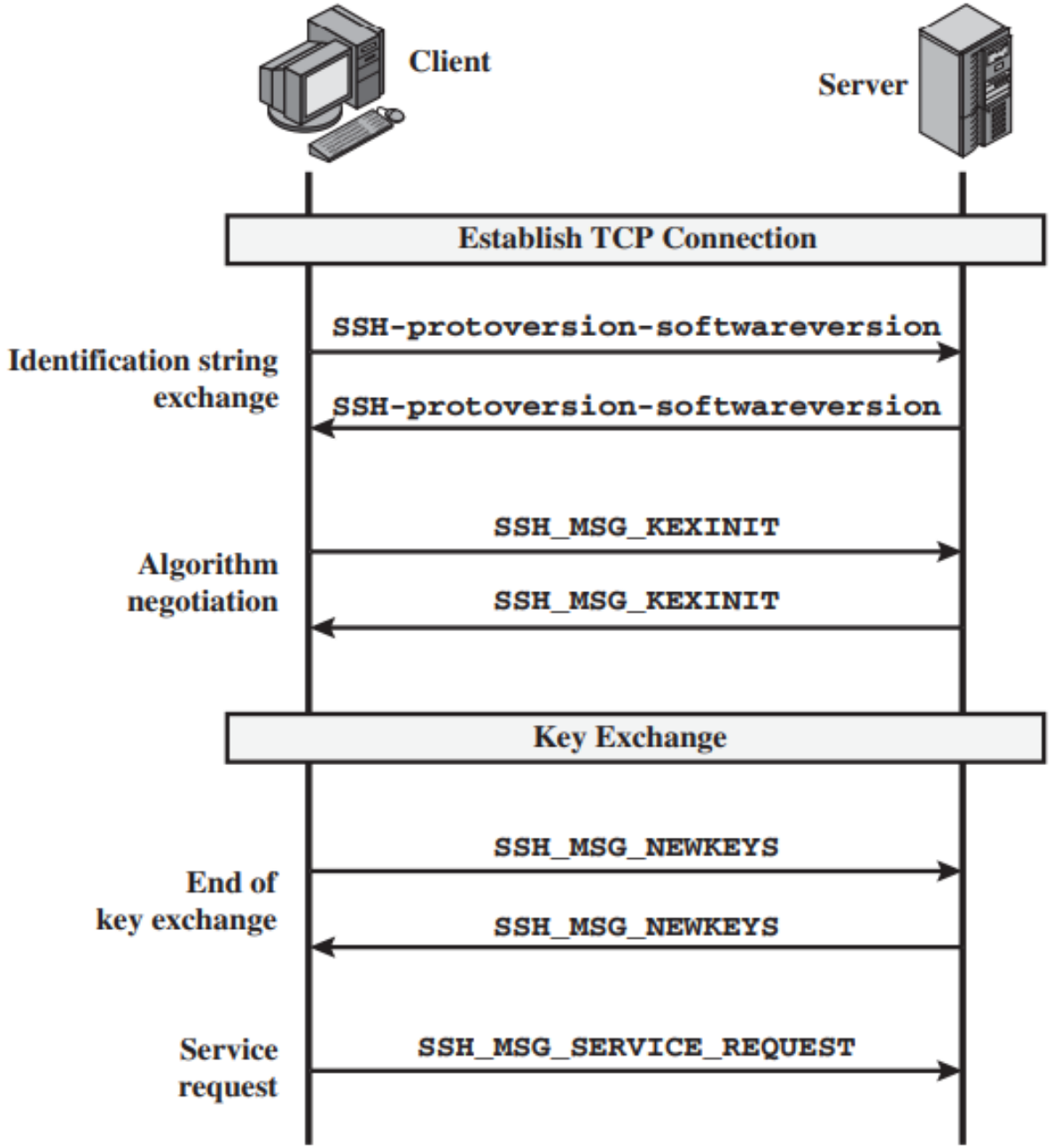

Figure 16.9 SSH Transport Layer Protocol Packet Exchanges

1. Identification String Exchange

- Begins with the client sending a packet with an identification string of the form:

```
SSH-protoversion-softwareversion SP comments CR LF
```

where SP, CR, and LF are space character, carriage return, and line feed, respectively. An example of a valid string is SSH-2.0-billsSSH_3.6.3q3<CR><LF>. The

- The server responds with its own identification string.
- These strings are used in the DiffieHellman key exchange.

2. Algorithm negotiation

- Each side sends an SSH_MSG_KEXINIT containing lists of supported algorithms in the order of preference to the sender.
- There is one list for each type of cryptographic algorithm.
- The algorithms include **key exchange, encryption, MAC algorithm, and compression algorithm**.

Table 16.3 SSH Transport Layer Cryptographic Algorithms

Cipher	
`3des-cbc`*	Three-key 3DES in CBC mode
`blowfish-cbc`	Blowfish in CBC mode
`twofish256-cbc`	Twofish in CBC mode with a 256-bit key
`twofish192-cbc`	Twofish with a 192-bit key
`twofish128-cbc`	Twofish with a 128-bit key
`aes256-cbc`	AES in CBC mode with a 256-bit key
`aes192-cbc`	AES with a 192-bit key
`aes128-cbc`**	AES with a 128-bit key
`Serpent256-cbc`	Serpent in CBC mode with a 256-bit key
`Serpent192-cbc`	Serpent with a 192-bit key
`Serpent128-cbc`	Serpent with a 128-bit key
`arcfour`	RC4 with a 128-bit key
`cast128-cbc`	CAST-128 in CBC mode

MAC algorithm	
`hmac-sha1`*	HMAC-SHA1; digest length = key length = 20
`hmac-sha1-96`**	First 96 bits of HMAC-SHA1; digest length = 12; key length = 20
`hmac-md5`	HMAC-SHA1; digest length = key length = 16
`hmac-md5-96`	First 96 bits of HMAC-SHA1; digest length = 12; key length = 16

Compression algorithm	
`none`*	No compression
`zlib`	Defined in RFC 1950 and RFC 1951

* = Required

** = Recommended

3. KEY EXCHANGE

- Two versions of Diffie-Hellman key exchange are specified.
- Both versions are defined in RFC 2409 and require only one packet in each direction.
- In the algorithm
  - ***C*** *is the client*
  - ***S*** *is the server*
  - ***p*** *is a large safe prime*
  - ***g*** *is a generator for a subgroup of* ***GF(p)****;*
  - ***q*** *is the order of the subgroup;*
  - ***V_S*** *is S's identification string;*
  - ***V_C*** *is C's identification string;*
  - ***K_S*** *is S's public host key;*
  - ***I_C*** *is C's SSH_MSG_KEXINIT message and* ***I_S*** *is S's SSH_MSG_KEXINIT message that have been exchanged before this part begins.*
- The values of p, g, and q are known to both client and server as a result of the algorithm selection negotiation.
- The hash function hash() is also decided during algorithm negotiation.

1. C generates a random number $x (1 < x < q)$ and computes $e = g^x \bmod p$. C sends e to S.
2. S generates a random number $y (0 < y < q)$ and computes $f = g^y \bmod p$. S receives e. It computes $K = e^y \bmod p$, $H = \text{hash}(\text{V\_C} \| \text{V\_S} \| \text{I\_C} \| \text{I\_S} \| \text{K\_S} \| e \| f \| \text{K})$, and signature s on H with its private host key. S sends $(\text{K\_S} \| f \| s)$ to C. The signing operation may involve a second hashing operation.

3. C verifies that K_S really is the host key for S (e.g., using certificates or a local database). C is also allowed to accept the key without verification; however, doing so will render the protocol insecure against active attacks (but may be desirable for practical reasons in the short term in many environments). C then computes $K = f^x \bmod p$, $H = \text{hash}(\text{V\_C} \| \text{V\_S} \| \text{I\_C} \| \text{I\_S} \| \text{K\_S} \| e \| f \| K)$, and verifies the signature s on H.

- As a result of these steps, the two sides now share a master key K.
- In addition, the server has been authenticated to the client, because the server has used its private key to sign its half of the Diffie-Hellman exchange.
- Finally, **the hash value H serves as a session identifier for this connection**.
- Once computed, **the session identifier is not changed**, even if the key exchange is performed again for this connection to obtain fresh keys.

4. Finalizing Key Exchange

- The end of key exchange is signaled by the exchange of SSH_MSG_NEWKEYS packets.
- At this point, both sides may start using the keys generated.

3. SERVICE REQUEST

- The final step is service request.
- The client sends an SSH_MSG_ SERVICE_REQUEST packet to request either the User Authentication or the Connection Protocol.
- Subsequent to this, all data is exchanged as the payload of an SSH Transport Layer packet, protected by encryption and MAC.

SSH Packet Format

Each SSH packet contains:

- **Packet Length**: Excludes itself & MAC field.
- **Padding Length**: Length of random padding.
- **Payload**: Main data (can be compressed).
- **Random Padding**: Ensures packet size matches cipher block size.
- **MAC (Message Authentication Code)**: Protects data integrity.